

Permisos de Archivos y Directorios en Linux

Introducción a los Permisos en Linux

En Linux, cada archivo y directorio tiene un conjunto de permisos que determinan quién puede leerlo, escribir en él o ejecutarlo. Esto es crucial para la seguridad, ya que evita que usuarios no autorizados accedan o modifiquen información que no les pertenece.

Los permisos se organizan en tres categorías principales de "usuarios":

1. **Propietario (Owner - u):** El usuario que creó el archivo o directorio. Solo puede haber un propietario.
2. **Grupo (Group - g):** Un grupo de usuarios. El propietario puede pertenecer a este grupo o no. Varios usuarios pueden pertenecer a un mismo grupo.
3. **Otros (Others - o):** Todos los demás usuarios del sistema que no son el propietario ni pertenecen al grupo.

Además de estas categorías, existe un cuarto "usuario" especial:

- **Todos (All - a):** Se refiere a propietario, grupo y otros a la vez.

Tipos de Permisos

Para cada una de las categorías anteriores (propietario, grupo, otros), se pueden asignar tres tipos de permisos:

1. **Lectura (Read - r):**
 - **En archivos:** Permite ver el contenido del archivo.
 - **En directorios:** Permite listar el contenido del directorio (ver qué archivos hay dentro).
2. **Escritura (Write - w):**
 - **En archivos:** Permite modificar o eliminar el archivo.
 - **En directorios:** Permite crear, eliminar o renombrar archivos dentro de ese directorio (incluso si no eres propietario de los archivos).
3. **Ejecución (Execute - x):**
 - **En archivos:** Permite ejecutar el archivo (si es un programa o un script).
 - **En directorios:** Permite "entrar" al directorio, es decir, navegar a él con `cd` y acceder a sus subdirectorios y archivos (si se tienen los permisos de lectura o escritura).

adecuados). Sin permiso de ejecución, aunque tengas lectura, no podrás ver el contenido.

Visualizando los Permisos: **ls -l**

Para ver los permisos de un archivo o directorio, usamos el comando `ls` con la opción `-l`.

Ejemplo:

```
ls -l mi_archivo.txt
```

Salida de ejemplo:

```
-rw-r--r-- 1 usuario1 grupo1 0 Fch_Mod Hora_Mod mi_archivo.txt
```

1. **Primer Carácter (-):** Indica el tipo de archivo.

- **-**: Archivo regular.
- **d**: Directorio.
- **l**: Enlace simbólico (symlink).
- **c**: Dispositivo de carácter.
- **b**: Dispositivo de bloque.

2. **Siguientes Nueve Caracteres (rw-r--r--):** Se dividen en tres grupos de tres, representando los permisos para:

- **Propietario (rw-)**: Lectura (r), Escritura (w), Sin ejecución (-).
- **Grupo (r--)**: Lectura (r), Sin escritura (-), Sin ejecución (-).
- **Otros (r--)**: Lectura (r), Sin escritura (-), Sin ejecución (-).

En este ejemplo, `mi_archivo.txt` puede ser leído y escrito por `usuario1` (el propietario), y solo leído por los miembros de `grupo1` y por cualquier otro usuario.

Gestión de Permisos: El comando **chmod**

El comando `chmod` (change mode) se utiliza para cambiar los permisos de archivos y directorios. Hay dos formas principales de especificar los permisos:

1. **Notación Simbólica (Symbolic Mode):** Más intuitiva, usando letras (u, g, o, a) y operadores (+, -, =).
2. **Notación Octal (Numeric Mode):** Más compacta, usando números para representar las combinaciones de permisos.

1. Notación Simbólica:

- **Quién:** **u** (propietario), **g** (grupo), **o** (otros), **a** (todos).
- **Operador:**
 - **+**: Añadir permiso.
 - **-**: Quitar permiso.
 - **=**: Asignar permisos exactos (reemplaza los existentes).
- **Permiso:** **r** (lectura), **w** (escritura), **x** (ejecución).

Ejemplos de chmod (Notación Simbólica):

1. Dar permiso de ejecución al propietario de mi_script.sh:

```
chmod u+x mi_script.sh
```

```
ls -l mi_script.sh          # Verás algo como -rwx r--r--
```

- Antes: -rw-r--r-- (si solo tenía lectura y escritura)
- Después: -rwxr--r--

2. Quitar permiso de escritura a otros en mi_archivo.txt:

```
chmod o-w mi_archivo.txt
```

```
ls -l mi_archivo.txt        # Verás algo como -rw-r--r--
```

- Antes: -rw-r--rw- (si tenía escritura para otros)
- Después: -rw-r--r--

3. Dar permisos de lectura y escritura al grupo, y solo lectura a otros en documento.pdf:

```
chmod g+rw,o+r documento.pdf
```

```
ls -l documento.pdf        # Verás algo como -rw-rw-r--
```

4. Establecer permisos exactos: propietario lectura/escritura/ejecución, grupo lectura/ejecución, otros solo lectura en mi_directorio/:

```
chmod u=rwx,g=rx,o=r mi_directorio/
```

```
ls -l mi_directorio/        # Verás algo como drwxr-xr--
```

2. Notación Octal

Cada permiso (r, w, x) tiene un valor numérico vinculado a su peso binario:

- **r (lectura): 4**
- **w (escritura): 2**
- **x (ejecución): 1**
- **- (sin permiso): 0**

Utilizando el sistema de numeración Octal (binario de 3 dígitos) obtenemos un número de 0 a 7 para todas las posibles combinaciones.

Permisos	rwX	Suma
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwX	111	7

Un permiso completo en notación octal se compone de tres dígitos, uno para cada categoría (propietario, grupo, otros).

Ejemplos de chmod (Notación Octal):

1. Permisos 755 (Típico para directorios y ejecutables):

- Propietario: rwx (4+2+1 = 7)
- Grupo: r-x (4+0+1 = 5)
- Otros: r-x (4+0+1 = 5)

```
chmod 755 mi_directorio/
```

```
ls -l mi_directorio/
```

Verás drwxr-xr-x

2. Permisos 644 (Típico para archivos de texto):

- Propietario: rw- (4+2+0 = 6)
- Grupo: r-- (4+0+0 = 4)
- Otros: r-- (4+0+0 = 4)

```
chmod 644 mi_archivo.txt
```

```
ls -l mi_archivo.txt          # Verás -rw-r--r--
```

3. Dar al propietario todos los permisos, al grupo lectura/escritura y a otros ningún permiso (760):

```
chmod 760 secreto.txt
```

```
ls -l secreto.txt            # Verás -rwxrw----
```

Opción -R (Recursivo):

Para cambiar permisos de un directorio y todo su contenido (subdirectorios y archivos), usa la opción -R.

```
chmod -R 755 mi_proyecto/
```

¡CUIDADO! Usar -R incorrectamente puede dejar tu sistema inestable o inseguro.

Gestión de Propietario y Grupo: chown y chgrp

Además de los permisos, también podemos cambiar quién es el propietario de un archivo o a qué grupo pertenece.

1. *chown* (Change Owner)

El comando `chown` cambia el propietario de un archivo o directorio. Solo el usuario `root` (o con `sudo`) puede usar este comando.

Sintaxis básica:

`chown [nuevo_propietario]:[nuevo_grupo] archivo_o_directorio`

- `[nuevo_propietario]`: El nombre del usuario al que se le asignará la propiedad.
- `[nuevo_grupo]`: (Opcional) El nombre del grupo al que se le asignará el archivo. Se puede omitir si solo quieres cambiar el propietario. Si usas ":" seguido de un grupo, cambia también el grupo (te ahorrarías usar después el *chgrp*)

Ejemplos de `chown`:

1. Cambiar el propietario de `documento.txt` a `usuario_nuevo`:

```
sudo chown usuario_nuevo documento.txt
```

2. Cambiar el propietario de `proyecto/` a `admin` y el grupo a `desarrollo`:

```
sudo chown admin:desarrollo proyecto/
```

3. Cambiar solo el propietario de `config.ini` a `root`, manteniendo el grupo actual:

```
sudo chown root config.ini
```

4. Cambiar el grupo de `script.sh` a `developers` sin cambiar el propietario:

```
sudo chown :developers script.sh
```

```
sudo chgrp developers script.sh
```

Opción `-R` (Recursivo):

```
sudo chown -R usuario_nuevo:grupo_nuevo mi_carpeta/
```

2. *chgrp* (Change Group)

El comando `chgrp` cambia el grupo propietario de un archivo o directorio. A diferencia de `chown`, un usuario normal puede usar `chgrp` para un archivo si es el propietario y el grupo al que quiere cambiar también pertenece a los grupos a los que él pertenece. Para otros casos, se requiere `root` (o `sudo`).

Sintaxis básica:

`chgrp [nuevo_grupo] archivo_o_directorio`

Ejemplos de chgrp:

1. **Cambiar el grupo de informe.odt a contabilidad:**

```
sudo chgrp contabilidad informe.odt
```

2. **Cambiar recursivamente el grupo de un directorio a web_dev:**

```
sudo chgrp -R web_dev /var/www/html
```

La Máscara de Permisos por Defecto: *umask*

Cuando un usuario crea un nuevo archivo o directorio, no se le asignan automáticamente los permisos más permisivos (como 777 para directorios o 666 para archivos). En su lugar, el sistema aplica una máscara de permisos por defecto, conocida como **umask**.

El umask es un valor numérico (en notación octal) que **resta** permisos a los permisos máximos posibles para un nuevo archivo o directorio.

- **Permisos máximos para archivos: 666** (rw-rw-rw-). Los archivos nunca tienen el permiso de ejecución por defecto por seguridad.
- **Permisos máximos para directorios: 777** (rwxrwxrwx).

El umask especifica qué permisos **no** se van a dar por defecto.

Cómo funciona umask:

1. El sistema toma los permisos máximos posibles (666 para archivos, 777 para directorios).
2. Aplica el umask restando esos bits de permiso a los permisos máximos.

Valor típico de umask en Ubuntu:

- **0022 (o simplemente 022):** Es muy común.
 - **Primer dígito 0:** Indica que no se usan permisos especiales (SUID, SGID, Sticky Bit).
 - **022:**
 - **0 para propietario:** $7 - 0 = 7$ (rwx)
 - **2 para grupo:** $7 - 2 = 5$ (r-x)
 - **2 para otros:** $7 - 2 = 5$ (r-x)

Con un umask de 022:

- **Nuevos directorios:** Obtendrán 755 (rwxr-xr-x).
 - $777 \text{ (máximo)} - 022 \text{ (umask)} = 755$.
- **Nuevos archivos:** Obtendrán 644 (rw-r--r--).
 - $666 \text{ (máximo)} - 022 \text{ (umask)} = 644$.
- **0002 (o simplemente 002):** También se usa, especialmente en entornos multiusuario donde los miembros del mismo grupo necesitan más permisos.
 - **002:**
 - **0 para propietario:** $7 - 0 = 7$ (rwx)
 - **0 para grupo:** $7 - 0 = 7$ (rwx)
 - **2 para otros:** $7 - 2 = 5$ (r-x)

Con un umask de 002:

- **Nuevos directorios:** Obtendrán 775 (rwxrwxr-x).
- **Nuevos archivos:** Obtendrán 664 (rw-rw-r--).

Comprobando la umask actual

Para ver el valor de umask actual de tu sesión, simplemente escribe umask en la terminal:

```
umask
```

Salida de ejemplo: 0022

Modificando la umask

Puedes cambiar la umask de tu sesión actual con el comando umask seguido del nuevo valor octal. Ten en cuenta que este cambio es **temporal** y solo afecta a la sesión actual de la terminal.

Ejemplos de umask:

1. Establecer un umask más restrictivo (077):

Esto hará que los nuevos archivos y directorios solo tengan permisos para el propietario.

- $777 - 077 = 700$ para directorios.
- $666 - 077 = 600$ para archivos.

```
umask 077
```

```
mkdir mi_secreto
```

```
touch archivo_secreto.txt
```



```
ls -l | grep "mi_secreto\|archivo_secreto"
```

- drwx----- (para mi_secreto)
- -rw----- (para archivo_secreto.txt)

2. Establecer un umask que permita al grupo escritura (002):

```
umask 002
```

```
mkdir compartido
```

```
touch archivo_grupo.txt
```

```
ls -l | grep "compartido\|archivo_grupo"
```

- drwxrwxr-x (para compartido)
- -rw-rw-r-- (para archivo_grupo.txt)

Para hacer un cambio de umask permanente:

La umask se configura normalmente en los archivos de configuración del shell, como:

- /etc/profile (para todos los usuarios, al iniciar sesión).
- /etc/bash.bashrc (para todos los usuarios, al iniciar bash).
- ~/.bashrc (para un usuario específico, al iniciar bash).
- ~/.profile (para un usuario específico, al iniciar sesión).

Un umask por defecto en Ubuntu suele estar definido en /etc/login.defs o en los scripts de inicio del shell, a menudo con lógica condicional (por ejemplo, 002 para usuarios con su propio grupo y 022 para usuarios que comparten el grupo users o staff).

Importancia de umask:

- **Seguridad:** Permite establecer una política de seguridad por defecto, asegurando que los archivos y directorios no sean excesivamente permisivos al ser creados, reduciendo el riesgo de accesos no autorizados.
- **Consistencia:** Mantiene una consistencia en los permisos de los nuevos elementos, lo cual es vital en entornos multiusuario o en servidores.
- **Gestión de Proyectos:** En un equipo, un umask bien configurado (ej. 002) puede facilitar la colaboración, permitiendo que todos los miembros del grupo del proyecto tengan permisos de escritura en los archivos compartidos por defecto.

Ejercicio de repaso resuelto:

Crea un directorio llamado PRUEBAPERMISOS en tu home y a continuación:

1. Crea un archivo de texto: `touch documento.txt`
2. Muestra sus permisos: `ls -l documento.txt`
3. Quita el permiso de lectura y escritura para otros: `chmod o-rw documento.txt`
4. Intenta leerlo como otro usuario (requiere cambiar de usuario o usar `sudo -u otro_usuario cat documento.txt`).
5. Crea un script: `echo "echo Hola mundo" > mi_script.sh`
6. Intenta ejecutarlo: `./mi_script.sh` (fallará porque no tiene permiso de ejecución).
7. Dale permiso de ejecución al propietario: `chmod u+x mi_script.sh`
8. Ejecútalo de nuevo: `./mi_script.sh` (ahora debería funcionar).
9. Crea un directorio: `mkdir carpeta_compartida`
10. Intenta cambiar su propietario a root: `sudo chown root carpeta_compartida`
11. Cambia sus permisos a 777 (¡solo para fines de demostración, generalmente inseguro!): `chmod 777 carpeta_compartida`
12. Explica por qué esto es peligroso en un entorno real.